

Análisis y rediseño del sistema de excepciones

OpenMarkov 0.3.0-SNAPSHOT

Informe técnico — Marzo 2026

Inventario completo de las excepciones del proyecto,
análisis detallado de los problemas identificados y sus consecuencias,
diagramas UML de la jerarquía actual y de la jerarquía rediseñada,
y plan de migración por fases.

Nota sobre los diagramas PlantUML

Los diagramas de este documento se generan con PlantUML a partir de los ficheros `exception-hierarchy-current.puml` y `exception-hierarchy-proposed.puml`.

Para regenerarlos: `java -jar plantuml.jar *.puml -tpng`

O con `plantuml` si está disponible en el PATH.

0 Índice

1. Qué es una excepción y por qué su diseño importa	3
2. Estado actual del sistema	3
2.1. Inventario	3
2.2. Patrón de clases internas	4
2.3. Interfaces de localización	4
2.4. Diagrama de la jerarquía actual	4
3. Problemas identificados	6
3.1. El exceso de excepciones comprobadas que nadie puede resolver	6
3.1.1. Descripción del problema	6
3.1.2. Qué ocurre en la práctica	6
3.1.3. Las 19 excepciones candidatas a <i>unchecked</i>	7
3.2. La firma genérica <code>throws Exception</code>	7
3.2.1. Descripción del problema	7
3.3. El silenciamiento silencioso: <code>catch → return null</code>	8
3.3.1. Descripción del problema	8
3.3.2. Por qué es peligroso	8
3.4. Conflictos de nombres entre clases internas	8
3.4.1. Descripción del problema	8
3.4.2. Por qué es peligroso	9
3.5. Errores ortográficos en nombres de clase	9
3.5.1. Descripción del problema	9
3.5.2. Por qué importa	9
3.6. Detalles técnicos internos expuestos en las APIs públicas	9
3.6.1. Descripción del problema	9
3.6.2. Por qué es peligroso	10
3.7. Excepciones <i>checked</i> que cruzan capas incorrectas	10
3.7.1. Descripción del problema	10
3.7.2. Qué significa cruzar capas incorrectas	10
3.8. Las excepciones de navegación de evidencia	11
3.8.1. Descripción del problema	11
3.8.2. Consecuencia	11
3.9. Pérdida del rastro del error al envolver <code>DoEditException</code>	11
3.9.1. Descripción del problema	11
3.9.2. Qué se pierde	12
3.10. Re-lanzamiento vacío en <code>ProbNet.setNetworkType()</code>	12
3.10.1. Descripción del problema	12
3.10.2. Por qué es un problema	12
4. Propuestas de rediseño	14
4.1. Propuesta A — Convertir a <i>unchecked</i> las excepciones no recuperables	14
4.2. Propuesta B — Reducir la superficie de excepciones <i>checked</i> a las verdaderamente recuperables	14
4.3. Propuesta C — Eliminar <code>throws Exception</code> genérica en las APIs de aprendizaje	15
4.4. Propuesta D — Corregir el antipatrón <code>catch → null</code> en <code>FormatManager</code>	15
4.5. Propuesta E — Nueva jerarquía semántica	15
4.5.1. Qué aporta la nueva jerarquía	16
4.6. Propuesta F — Corrección de typos y conflictos de nombres	18

4.7. Propuesta G — Ocultar excepciones de infraestructura en las APIs de I/O	18
4.8. Propuesta H — Corrección de la pérdida de traza en <code>DoEditException</code>	18
4.9. Propuesta I — Eliminación del re-lanzamiento vacío en <code>setNetworkType()</code>	19
4.10. Propuesta J — Revisión de la lógica de navegación de evidencia	19
5. Plan de migración	20

1 Qué es una excepción y por qué su diseño importa

En cualquier programa Java, una **excepción** es el mecanismo formal mediante el cual una pieza de código avisa a sus llamadores de que algo ha salido mal y de que no puede continuar normalmente. Cuando ocurre un problema, el código «lanza»(en inglés, *throws*) una excepción; el código que llamó al primero tiene entonces la oportunidad de «capturarla»(en inglés, *catch*) y tomar alguna medida correctiva, o bien dejar que siga propagándose hacia capas más externas del programa.

El lenguaje Java establece una distinción fundamental entre dos tipos de excepciones:

Excepciones comprobadas (*checked*) Son aquellas que Java obliga a declarar explícitamente en la firma de cada método que puede lanzarlas. Cualquier método que llame a otro que declare una excepción comprobada tiene dos únicas opciones legales: capturarla allí mismo, o declarar a su vez que él también puede lanzarla. Esto crea una *cadena de responsabilidad formal* a lo largo de toda la pila de llamadas. El objetivo original de este mecanismo es obligar al programador a pensar en los errores recuperables: aquellos ante los cuales el llamador puede hacer algo útil (mostrar un mensaje al usuario, intentar una alternativa, etc.).

Excepciones no comprobadas (*unchecked*) Son aquellas que representan errores de programación o estados internamente imposibles: un puntero nulo, un índice fuera de rango, una suposición de diseño que se ha violado. Java *no* obliga a declararlas ni a capturarlas. Se propagan automáticamente hasta que alguien las atrapa (normalmente en la capa más externa del programa, que las muestra como un error interno) o hasta que terminan el programa con un volcado de pila de llamadas.

El diseño adecuado del sistema de excepciones de una aplicación es un **contrato de comunicación entre capas**: indica claramente qué puede fallar de forma recuperable, qué representa un error de programación y qué simplemente no puede ocurrir en condiciones normales. Un sistema de excepciones mal diseñado deteriora ese contrato, hace que los errores sean invisibles o difíciles de diagnosticar, y aumenta significativamente el coste de mantenimiento.

2 Estado actual del sistema

2.1 Inventario

El proyecto define **62 clases de excepción propias**, distribuidas entre seis módulos. Casi todas son comprobadas (*checked*), salvo dos (`UnreachableException` y `UnrecoverableException`, que son *unchecked*).

Módulo	Paquete principal	Clases
core	org.openmarkov.core.exception	27
gui	org.openmarkov.gui.exception	27
io	org.openmarkov.io.probmodel.exception	1
learning.core	org.openmarkov.learning.core.exception	2
learning.gui	org.openmarkov.learning.exception	2
stochasticPropagationOutput	(módulo propio)	1
Total		62

2.2 Patrón de clases internas

Doce excepciones actúan como *namespaces* de variantes, agrupando múltiples situaciones de error bajo una excepción base común mediante clases internas estáticas:

Excepción base	Variantes internas
ConstraintViolatedException	42
ParserException	15
PGMXParserException	14
NonProjectablePotentialException	8
WriterException	6
DoEditException	5
CostEffectivenessException	4
IncompatibleEvidenceException	3
NotEvaluableNetworkException	3
InvalidNetworkTypeException	2
LearningException	2
PotentialOperationException	1

2.3 Interfaces de localización

Existe un sistema dual de interfaces:

- `IOpenMarkovException` — interfaz base para todas las excepciones propias; incluye métodos para obtener el mensaje y el título de la excepción.
- `IBundledOpenMarkovException` — extiende la anterior; añade soporte para mensajes localizados en ficheros `.properties`, de forma que los diálogos de error puedan mostrarse en el idioma del usuario.

No todas las excepciones implementan estas interfaces de forma consistente.

2.4 Diagrama de la jerarquía actual

Las figuras 1 y 2 muestran la jerarquía de excepciones tal como está implementada en la versión 0.3.0-SNAPSHOT. Las excepciones sombreadas en **rojo** son aquellas que el propio código fuente identifica, mediante comentarios `TODO`, como candidatas a convertirse en *unchecked*. Las sombreadas en **naranja** no habían sido analizadas en ningún informe previo sobre el sistema de excepciones.

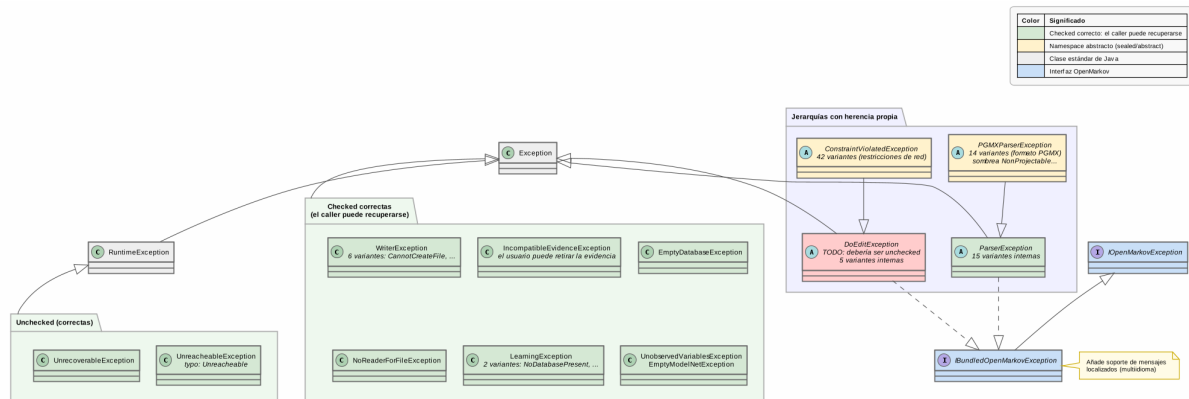


Figura 1: Jerarquía de excepciones actual (parte 1): excepciones correctamente diseñadas y jerarquías con herencia propia. Fuente: `exception-hierarchy-current-a.puml`.

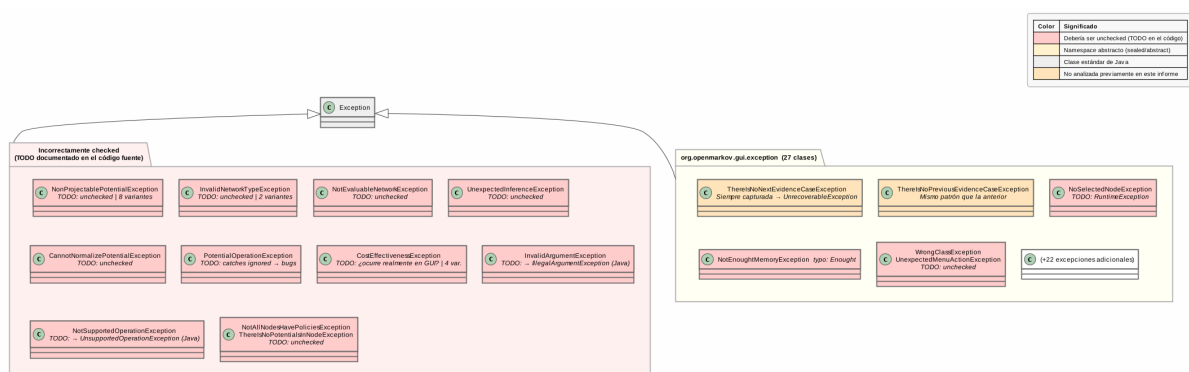


Figura 2: Jerarquía de excepciones actual (parte 2): excepciones incorrectamente declaradas como *checked* y excepciones del módulo GUI. Fuente: `exception-hierarchy-current-b.puml`.

3 Problemas identificados

3.1 El exceso de excepciones comprobadas que nadie puede resolver

3.1.1 Descripción del problema

En Java, declarar una excepción como *checked* es una promesa al llamador: «Si me llamas, es posible que ocurra este error, y *tienes* alguna forma de manejarlo.» El mecanismo tiene sentido cuando el llamador dispone de una alternativa real: si no se puede abrir un fichero, puede pedir al usuario que indique otro; si la red no tiene nodos, puede mostrar un aviso y detener la operación con elegancia.

Sin embargo, OpenMarkov declara como comprobadas **19 excepciones ante las cuales ningún llamador puede hacer nada útil**. El propio código fuente lo reconoce explícitamente: cada uno de esos 19 ficheros contiene un comentario `TODO` admitiendo que la excepción debería ser *unchecked*. Los siguientes son extractos literales de esos comentarios:

- `DoEditException`: «*Either design is poor, or the exceptions it encloses could be turned into RuntimeExceptions.*»
- `NonProjectablePotentialException`: «*Almost every exception of this class is wrapped into an UnreachableException or shown with JOptionPane. This might be a RuntimeException.*»
- `NotSupportedOperationException`: «*Exceptions of this type are always caught and then ignored. Perhaps they should be turned into RuntimeExceptions.*»

3.1.2 Qué ocurre en la práctica

Cuando un programador llama a un método que declara una excepción *checked* que sabe que no puede manejar, tiene dos opciones, ninguna de ellas buena:

1. **Capturarla y no hacer nada**: el código atrapa la excepción, ignora el error, y el programa continúa en un estado posiblemente inconsistente. El usuario no ve ningún aviso; el desarrollador no recibe ninguna traza. Este patrón, conocido como *excepción tragada* (*swallowed exception*), es una de las principales causas de errores silenciosos difíciles de reproducir.
2. **Envolverla en una excepción *unchecked* y relanzarla**: el código atrapa la excepción, la mete dentro de una *unchecked* (en OpenMarkov, `UnreachableException` o `UnrecoverableException`), y la lanza de nuevo. El resultado final es idéntico a haber declarado la excepción como *unchecked* desde el principio, pero con el coste de haber obligado a *todos* los métodos intermedios de la pila de llamadas a declarar o capturar la excepción original.

En OpenMarkov, el método `MainPanelListenerAssistant.actionPerformed()` —el punto central de control de la interfaz gráfica— captura un bloque de **siete excepciones distintas** más de veinte veces a lo largo de sus 380 líneas, relanzándolas todas como `UnrecoverableException`. Ese bloque se repite prácticamente idéntico en cada acción. La causa directa es que los métodos invocados declaran excepciones comprobadas que la GUI no puede resolver; el resultado es 200+ líneas de código de gestión de errores que no gestiona nada, solo propaga.

3.1.3 Las 19 excepciones candidatas a unchecked

Excepción	Argumento del TODO en el código
<code>DoEditException</code>	El diseño es deficiente o las excepciones que encapsula podrían ser <i>unchecked</i>
<code>IncompatibleEvidenceException</code>	Capturada e ignorada en casi todos los bloques <code>catch</code> , lo que probablemente genera bugs inesperados
<code>NonProjectablePotentialException</code>	Casi siempre envuelta en <code>UnreachableException</code> o mostrada con <code>JOptionPane</code>
<code>NotEvaluableNetworkException</code>	Solo se captura para mostrar un mensaje genérico
<code>NotSupportedOperationException</code>	Siempre capturada y luego ignorada
<code>InvalidArgumentException</code>	Podría ser una <i>RuntimeException</i>
<code>UnexpectedInferenceException</code>	Las capturas siempre la envuelven con <code>UnreachableException</code>
<code>InvalidNetworkTypeException</code>	Siempre envuelta en <code>UnreachableException</code>
<code>CannotNormalizePotentialException</code>	¿Realmente ocurre en la GUI?
<code>CostEffectivenessException</code>	¿Realmente ocurre en la GUI?
<code>NotAllNodesHavePoliciesException</code>	Capturada y lanzada como <code>UnreachableException</code>
<code>PotentialOperationException</code>	Las capturas solo la muestran y la ignoran, generando bugs posteriores
<code>WrongClassException</code>	Probablemente debería ser <code>UnreachableException</code>
<code>UnexpectedMenuActionException</code>	Probablemente debería ser <code>UnreachableException</code>
<code>NoSelectedNodeException</code>	Probablemente debería ser <i>RuntimeException</i>
<code>IntervalsAreNotEvenException</code>	¿Realmente ocurre en la GUI?
<code>IntervalsAreNotMultipleOf3Exception</code>	¿Realmente ocurre en la GUI?
<code>NonProjectablePotentialException.CannotEvaluate</code>	Solo se usa una vez y se envuelve en <code>UnreachableException</code>
<code>ThereIsNoPotentialsInNodeException</code>	Capturada y relanzada como <i>unchecked</i>

3.2 La firma genérica throws Exception

3.2.1 Descripción del problema

Java permite declarar `throws Exception` en la firma de un método, lo que significa «este método puede lanzar cualquier excepción posible». Es el equivalente en el lenguaje de firmar un contrato en blanco: el llamador no sabe qué puede fallar ni bajo qué condiciones.

OpenMarkov tiene **82 firmas de método** que declaran `throws Exception` sin especificar tipo. La mayoría se concentran en el ciclo de aprendizaje automático (`LearningAlgorithm.getBestEdit()`, `LearningAlgorithm.getNextEdit()`) y en los *callbacks* de plugins.

Cuando un método declara `throws Exception`, el llamador se ve forzado a capturar `Exception` (la clase más genérica posible) o a declarar a su vez `throws Exception`, propagando el problema. Ningún compilador ni herramienta de análisis estático puede indicar al desarrollador qué situaciones de error debe contemplar realmente. En la práctica, los desarrolladores suelen capturar `Exception` con un bloque vacío o con un log genérico, perdiendo toda información sobre la causa real del fallo.

3.3 El silenciamiento silencioso: `catch` → `return null`

3.3.1 Descripción del problema

En el fichero `FormatManager.java` (líneas 63–78) existe un bloque que carga dinámicamente los plugins de lectura de formatos de archivo. Cuando un plugin no se puede instanciar correctamente (por ejemplo, porque le falta una clase en el classpath, o porque su constructor lanza una excepción), el código captura el error y devuelve `null`, sin registrar ningún mensaje en el log y sin avisar al sistema de que el plugin ha fallado:

```
1 // FormatManager.java, líneas 63-78
2 .map(readerClass -> {
3     try {
4         return readerClass.getDeclaredConstructor().newInstance();
5     } catch (InstantiationException | NoSuchMethodException
6             | IllegalAccessException | InvocationTargetException e) {
7         return null; // <-- error silenciado; el reader queda null
8     }
9 })
```

Listing 1: Antipatrón en `FormatManager.java` — error silenciado

3.3.2 Por qué es peligroso

Este patrón —conocido como *excepción tragada con retorno nulo* (*catch-and-return-null*)— introduce una **bomba de relojería** en el sistema. En el momento en que ocurre el error, el código no hace nada observable; la lista de plugins simplemente contiene un elemento `null` donde debería haber un lector funcional.

El fallo real no se manifiesta en ese punto sino *más adelante*, cuando algún otro código intenta usar ese `null` como si fuera un lector válido. En ese momento se produce una `NullPointerException` que no tiene ninguna relación visible con la causa original: el desarrollador verá un error en un lugar completamente distinto de aquel donde ocurrió el problema real, con una traza que no contiene ninguna referencia al plugin que falló ni al motivo por el que falló.

Si un usuario instala un plugin de formato de archivo mal empaquetado, OpenMarkov no le mostrará ningún aviso. El sistema simplemente ignorará ese formato. El usuario solo descubrirá el problema cuando intente abrir o guardar un fichero de ese tipo, y en ese momento el mensaje de error no le indicará que el plugin es el responsable.

3.4 Conflictos de nombres entre clases internas

3.4.1 Descripción del problema

Dos clases internas tienen el mismo nombre que otras excepciones ya existentes en el proyecto:

- `WriterException.NonProjectablePotentialException` tiene el mismo nombre que `org.openmarkov.core.exception.NonProjectablePotentialException`.
- `PGMXParserException.EvidenceIsIncompatibleWithOther` tiene el mismo nombre que `IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther`.

3.4.2 Por qué es peligroso

En Java, cuando se usa el nombre corto de una clase (sin el paquete completo) dentro de un fichero que tiene acceso a dos clases con ese mismo nombre, el compilador resuelve la ambigüedad usando la que está *más cerca* en el ámbito. Esto se denomina *sombreado de nombres* (*name shadowing*). El resultado práctico es que un desarrollador que escribe código en los módulos afectados puede estar usando *sin saberlo* la clase interna en lugar de la clase raíz, o viceversa.

En ambos casos citados, la clase interna y la clase raíz representan el mismo concepto de error, lo que confirma que se trata de código duplicado. Si en el futuro se modifica el comportamiento de la clase raíz (por ejemplo, añadiendo un nuevo campo de información de diagnóstico), la clase interna no recibirá esa mejora automáticamente, y el sistema presentará comportamientos inconsistentes dependiendo de qué excepción se lance en cada punto del código.

3.5 Errores ortográficos en nombres de clase

3.5.1 Descripción del problema

Ocho clases de excepción tienen errores tipográficos en su nombre. Los nombres de las clases en Java son parte de la API pública y, en el caso de las excepciones, aparecen directamente en los mensajes de error y en las trazas de pila que ven los desarrolladores.

Nombre actual (incorrecto)	Nombre correcto
UnsatisfiedContraints	UnsatisfiedConstraints
SamplesWeigthIsZero	SamplesWeightIsZero
MissingPropertiesOfContiousVariable	MissingPropertiesOfContinuousVariable
CannotAssignPotentialToStaticVariable	CannotAssignPotentialToStaticVariable
TriedToSplitInvervalOutsideBoundsException	TriedToSplitInterval- OutsideBoundsException
ValueOutOfDomaingRangeException	ValueOutOfDomainRangeException
NotEnoughMemoryException	NotEnoughMemoryException
UnreacheableException	UnreachableException

3.5.2 Por qué importa

Los nombres incorrectos de clase aparecen en: las trazas de pila de error que ven los desarrolladores, los registros de log, la documentación generada automáticamente (Javadoc), y los resultados de búsquedas en el código. Un desarrollador que busque `unreachable` no encontrará `UnreacheableException`. Un desarrollador que busque `constraint` no encontrará `UnsatisfiedContraints`. Los nombres con errores ortográficos actúan como *ruido* en el sistema de búsqueda e indexación del código.

3.6 Detalles técnicos internos expuestos en las APIs públicas

3.6.1 Descripción del problema

Varios métodos públicos de los módulos de I/O declaran explícitamente excepciones de las bibliotecas que usan internamente:

```
1 // Antes      API acoplada a la tecnolog a de implementaci n
2 public ProbNetReader getProbNetReader(URL url)
```

```
3 throws SAXException, IOException,  
4        NoReaderForFileException, ParserException;
```

Listing 2: API que expone detalles de implementación

`SAXException` es una excepción específica de la biblioteca SAX, que es la biblioteca de análisis de XML elegida internamente. `IOException` es una excepción de Java para operaciones de entrada/salida de bajo nivel.

3.6.2 Por qué es peligroso

Un llamador de este método —por ejemplo, la interfaz gráfica— tiene que importar y conocer `SAXException` aunque nunca haya decidido usar SAX para nada. Si en el futuro se decide cambiar la implementación del parser XML (por ejemplo, de SAX a StAX o a JDOM), *todos los llamadores del método* tendrán que actualizarse, aunque para ellos el cambio sea completamente transparente: su trabajo es «abrir una red», no «analizar XML con SAX». Este acoplamiento innecesario entre capas hace que cambios internos de implementación tengan un impacto externo mucho mayor del que deberían tener.

La regla general es que una API pública solo debe exponer excepciones definidas en términos del *dominio del problema*, no de los detalles de su implementación.

3.7 Excepciones *checked* que cruzan capas incorrectas

3.7.1 Descripción del problema

`DoEditException` está definida en el módulo `core` —la capa de dominio más baja— y aparece en **113 firmas de método** distribuidas por todos los módulos del proyecto: `gui`, `learning.algorithm`, `learning.gui`, etc.

3.7.2 Qué significa cruzar capas incorrectas

En una arquitectura bien estructurada, las dependencias entre capas fluyen en una sola dirección: desde las capas externas (GUI, aprendizaje) hacia las capas internas (dominio, modelo). Una excepción definida en la capa interna puede ser usada por las capas externas, pero no debería *obligar* a esas capas a declarar y gestionar detalles de la capa interna.

Cuando `DoEditException` —una excepción de la capa de dominio más básica— aparece en firmas de la GUI, obliga a la interfaz gráfica a conocer y gestionar el vocabulario de errores del motor de edición. Cualquier programador que trabaje en la GUI debe entender qué es un *edit* de dominio para poder manejar correctamente el error. Esto viola el principio de separación de responsabilidades: la GUI debería poder gestionar errores de UI sin necesidad de conocer los mecanismos internos del modelo de dominio.

En la práctica, los 113 métodos que declaran `DoEditException` en capas superiores no hacen nada útil con ella: simplemente la relanzan como `UnrecoverableException` o la muestran con un `JOptionPane`. El único efecto real es que todos esos métodos *tienen que importar* el paquete `org.openmarkov.core.exception`, creando dependencias de compilación innecesarias.

3.8 Las excepciones de navegación de evidencia: `ThereIsNoNextEvidenceCaseException` y `ThereIsNoPreviousEvidenceCaseException`

3.8.1 Descripción del problema

`ThereIsNoNextEvidenceCaseException` y `ThereIsNoPreviousEvidenceCaseException` son excepciones comprobadas del módulo `gui`. Se lanzan cuando el usuario intenta navegar (con los botones «caso siguiente» o «caso anterior») más allá del límite de la lista de casos de evidencia.

Ambas tienen una implementación mínima —sin campos adicionales, sin información contextual— y son siempre capturadas en `MainPanelListenerAssistant` para ser inmediatamente relanzadas como `UnrecoverableException`:

```

1  case GO_TO_NEXT_EVIDENCE_CASE -> {
2      try {
3          evidenceCasesNavigationOption("GO_TO_NEXT_EVIDENCE_CASE");
4      } catch (NotEvaluableNetworkException
5              | NonProjectablePotentialException
6              | NotEnoughMemoryException
7              | IncompatibleEvidenceException
8              | CannotNormalizePotentialException
9              | ThereIsNoNextEvidenceCaseException // <-- declarada
10             | ThereIsNoPreviousEvidenceCaseException // <-- declarada
11             | ConstraintViolatedException ex) {
12          throw new UnrecoverableException(ex); // relanzada sin
13              tratamiento
14      }
15  }
```

Listing 3: Patrón real de uso de estas excepciones en la GUI

Este mismo bloque de siete excepciones se repite *seis veces* para los seis botones de navegación (primero, anterior, siguiente, último, nuevo caso, borrar todo).

3.8.2 Consecuencia

Estas excepciones *podrían* ser útiles: saber que el usuario ha llegado al final de la lista de evidencias es información relevante que podría usarse para deshabilitar el botón «siguiente» antes de que el usuario lo pulse. Sin embargo, en la implementación actual, ese razonamiento no ocurre: la excepción se lanza *después* del intento fallido, se captura, y se relanza como error irrecoverable, lo que probablemente resulta en mostrar un mensaje de error al usuario en lugar de simplemente deshabilitar el botón.

La corrección no es solo cambiar el tipo de la excepción, sino repensar la lógica: el límite de la lista de evidencias debe consultarse *antes* de intentar navegar, para que los botones reflejen correctamente el estado.

3.9 Pérdida del rastro del error al envolver `DoEditException`

3.9.1 Descripción del problema

En `Node.java` (línea 807), cuando se captura una `NonProjectablePotentialException` y se envuelve en una `DoEditException`, se comete el siguiente error:

```

1 // Node.java, l nea ~807 MAL
2 } catch (NonProjectablePotentialException e) {
3     throw new DoEditException.CannotDoEditException(e.getMessage());
4     // solo el mensaje
5 }
6 // CORRECTO: pasar la excepci n original como causa
7 } catch (NonProjectablePotentialException e) {
8     throw new DoEditException.CannotDoEditException(e); // causa
9     completa
10 }

```

Listing 4: Pérdida del rastro de la excepción original en Node.java

3.9.2 Qué se pierde

Cuando se lanza una nueva excepción pasando únicamente `e.getMessage()` en lugar de `e` (la excepción completa), se pierde la *causa raíz* (*cause*). En términos prácticos, esto significa que la traza de pila (*stack trace*) que finalmente llega al log de la aplicación o al desarrollador **no contiene ninguna información sobre dónde se originó el error real**. Solo se ve el punto donde se lanzó la `DoEditException`, no el punto donde falló el cálculo de probabilidades que la provocó.

Imagine que un operario de una planta química reporta: «Ha habido un problema.» Sin indicar dónde, en qué equipo, ni bajo qué condiciones. El rastro es la diferencia entre un diagnóstico en segundos y una búsqueda que puede durar horas.

3.10 Re-lanzamiento vacío en ProbNet.setNetworkType()

3.10.1 Descripción del problema

En ProbNet.java (líneas 362–368) existe el siguiente bloque:

```

1 // ProbNet.java, l neas 362-368
2 public void setNetworkType(NetworkType networkType)
3     throws ConstraintViolatedException {
4     try {
5         // ... l gica de cambio de tipo
6     } catch (ConstraintViolatedException e) {
7         throw e; // relanza sin ning n tratamiento adicional
8     }
9 }

```

Listing 5: Re-lanzamiento vacío en ProbNet.setNetworkType()

El bloque `try/catch` captura `ConstraintViolatedException` y la relanza inmediatamente, sin agregar ningún mensaje, sin registrar ningún log, sin modificar la excepción en modo alguno.

3.10.2 Por qué es un problema

Un bloque `try/catch` que no realiza ningún tratamiento es código muerto con un coste: añade un nivel innecesario en la pila de llamadas (y por tanto en la traza de error), introduce ruido

visual en el código que dificulta entender el flujo real, y puede confundir a futuros desarrolladores haciéndoles creer que hay algún tratamiento implícito que en realidad no existe.

La excepción podría declararse directamente en la firma del método y dejar que se propagase sola, eliminando el `try/catch` sin ningún cambio de comportamiento.

4 Propuestas de rediseño

4.1 Propuesta A — Convertir a *unchecked* las excepciones no recuperables

Impacto: alto. Riesgo: medio.

Las 19 excepciones identificadas en el apartado 3.1 deben convertirse en *RuntimeException*. El criterio es preciso: si el llamador nunca puede hacer nada útil al capturar una excepción más que mostrar un mensaje de error genérico o relanzarla, esa excepción debe ser *unchecked*.

Las candidatas inmediatas son:

- `NonProjectablePotentialException` → nueva `PotentialComputationException` (*unchecked*)
- `NotEvaluableNetworkException` → *unchecked* (o *checked* justificada, ver Propuesta B)
- `InvalidArgumentException` → `IllegalArgumentException` (estándar de Java)
- `NotSupportedOperationException` → `UnsupportedOperationException` (estándar de Java)
- `InvalidNetworkTypeException` → *unchecked*
- `UnexpectedInferenceException` → *unchecked*
- `CannotNormalizePotentialException` → *unchecked*
- `NoSelectedNodeException` → *unchecked*
- `WrongClassException`, `UnexpectedMenuActionException` → *unchecked*

Convertir estas excepciones eliminará miles de declaraciones `throws` en cascada y simplificará radicalmente los bloques `try/catch` de `MainPanelListenerAssistant`, reduciendo ese método de 380 líneas en varios cientos de líneas.

4.2 Propuesta B — Reducir la superficie de excepciones *checked* a las verdaderamente recuperables

Impacto: alto. Riesgo: bajo.

Deben mantenerse como comprobadas *únicamente* las excepciones ante las cuales el llamador tiene una acción correctiva concreta y real:

Excepción	Acción correctiva posible para el llamador
<code>ParserException</code>	Mostrar mensaje descriptivo; ofrecer abrir otro fichero
<code>WriterException</code>	Reintentar en otra ruta; verificar permisos de escritura
<code>NoReaderForFileException</code>	Informar al usuario de que el formato no está soportado; sugerir instalar el plugin
<code>EmptyDatabaseException</code>	Pedir al usuario un fichero de datos con contenido
<code>IncompatibleEvidenceException</code>	Ofrecer al usuario la opción de retirar la evidencia conflictiva

<code>ConstraintViolatedException</code>	Mostrar qué restricción se ha violado y cómo corregirlo
<code>LearningException</code>	Indicar qué requisito de aprendizaje falta

Todo lo demás debe ser *unchecked*.

4.3 Propuesta C — Eliminar `throws Exception` genérica en las APIs de aprendizaje

Impacto: medio. Riesgo: bajo.

Los métodos `LearningAlgorithm.getBestEdit()` y `LearningAlgorithm.getNextEdit()` declaran `throws Exception`. La propuesta es:

1. Identificar qué excepciones concretas lanzan realmente todas las implementaciones existentes de estos métodos.
2. Declarar esas excepciones específicas en la firma; o bien,
3. Si todas son irre recuperables, convertirlas a *unchecked* y eliminar el `throws` por completo.

4.4 Propuesta D — Corregir el antipatrón `catch → null` en `FormatManager`

Impacto: bajo. Riesgo: muy bajo.

```

1 // ANTES
2 } catch (InstantiationException | NoSuchMethodException
3         | IllegalAccessException | InvocationTargetException e) {
4     return null; // error silenciado
5 }
6
7 // DESPU S
8 } catch (InstantiationException | NoSuchMethodException
9         | IllegalAccessException | InvocationTargetException e) {
10    throw new PluginConfigurationException(
11        "Cannot instantiate reader " + readerClass.getName(), e);
12 }
```

Listing 6: Corrección del antipatrón en `FormatManager.java`

Donde `PluginConfigurationException` es una nueva excepción *unchecked* que falla de forma inmediata y visible, con el contexto completo del error (nombre del plugin fallido y causa raíz).

4.5 Propuesta E — Nueva jerarquía semántica

Impacto: alto. Riesgo: alto.

La jerarquía actual organiza las excepciones por módulo (`core`, `gui`, `io`). La propuesta reorganiza por *naturaleza semántica*: qué tipo de situación representa cada excepción, independientemente de dónde se haya definido.

Las figuras 3 y 4 muestran la nueva jerarquía dividida en sus dos ramas principales. A continuación se detalla qué aporta cada nivel.

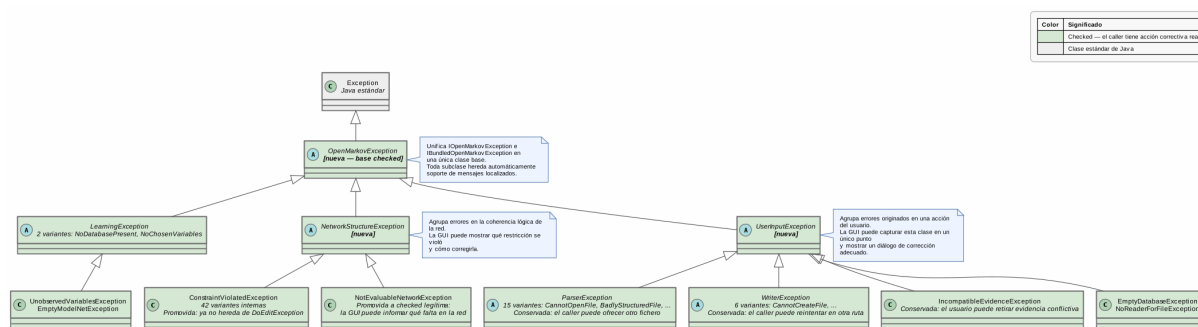


Figura 3: Jerarquía propuesta (parte 1): rama de excepciones *checked* bajo `OpenMarkovException`. Fuente: `exception-hierarchy-proposed-checked.puml`.

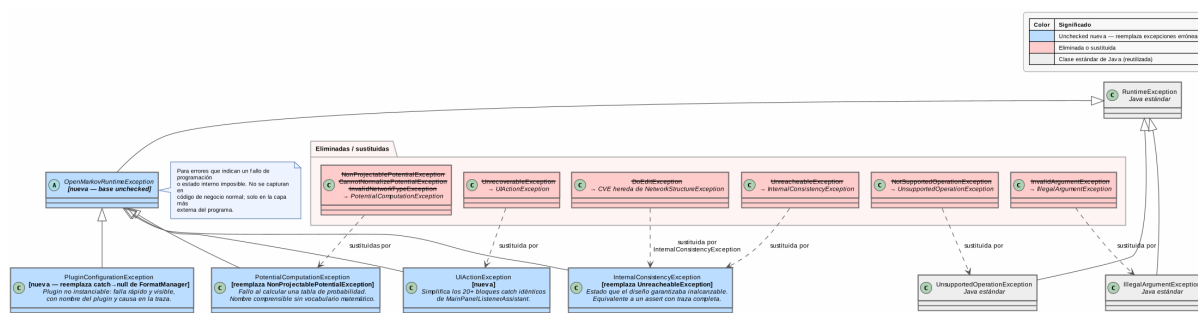


Figura 4: Jerarquía propuesta (parte 2): rama de excepciones *unchecked* bajo `OpenMarkovRuntimeException`, con las clases eliminadas o sustituidas. Fuente: `exception-hierarchy-proposed-unchecked.puml`.

4.5.1 Qué aporta la nueva jerarquía

La reorganización propuesta introduce dos ramas claras, separadas por criterio de uso, no de módulo:

OpenMarkovException (base de las excepciones comprobadas). Unifica las actuales interfaces `IOpenMarkovException` e `IBundledOpenMarkovException` en una única clase base concreta. Esto elimina la dualidad actual (dos interfaces, implementadas de forma inconsistente según la excepción) y garantiza que cualquier excepción comprobada de `OpenMarkov` tenga automáticamente soporte para mensajes localizados en el idioma del usuario.

UserInputException. Agrupa todos los errores que tienen su origen directo en una acción del usuario: abrir un fichero con formato incorrecto, intentar escribir en una ruta sin permisos, introducir evidencia contradictoria. La GUI puede capturar **UserInputException** en un único punto y mostrar un diálogo de corrección adecuado, sin necesidad de conocer si el error vino de un **ParserException** o de un **WriterException**.

NetworkStructureException. Agrupa los errores relacionados con la consistencia lógica de la red probabilística: restricciones violadas, tipo de red inválido para la operación solicitada, red que no puede evaluarse en el estado actual. Estos errores son recuperables: la GUI puede mostrar exactamente qué restricción se violó y guiar al usuario a corregirla. Al separarlos en una rama propia, el código de la GUI puede distinguir «el usuario ha hecho algo que viola una regla de la red»de «ha ocurrido un error interno inesperado».

OpenMarkovRuntimeException (base de las excepciones no comprobadas). Todos los errores que representan fallos de programación o estados imposibles según el diseño se unifican bajo esta base. Nadie en el código de negocio normal captura esta excepción; solo la capa más externa del programa la captura para registrar el error y mostrar un mensaje genérico al usuario.

InternalConsistencyException. Reemplaza a `UnreacheableException` (con el typo corregido). Se lanza cuando el programa alcanza un punto de código que el diseño garantizaba inalcanzable: por ejemplo, un valor de un `enum` que no está cubierto por un `switch`. En lugar de fallar con una excepción poco descriptiva, esta excepción incluye suficiente contexto para que un desarrollador identifique inmediatamente qué suposición de diseño se ha violado.

PotentialComputationException. Reemplaza a `NonProjectablePotentialException`. El nuevo nombre es más descriptivo para cualquier desarrollador que no conozca el término matemático «potencial proyectable». Siendo *unchecked*, elimina las decenas de declaraciones `throws` en cascada que actualmente impregnan el módulo de inferencia.

PluginConfigurationException. Excepción nueva que reemplaza el antipatrón `catch → null` de `FormatManager`. Al ser *unchecked*, se propaga automáticamente hasta la capa de gestión de errores de la aplicación, donde se registra con el contexto completo (nombre del plugin, causa original).

UIActionException. Excepción nueva. Permite reemplazar los más de veinte bloques `catch` idénticos en `MainPanelListenerAssistant` por un único wrapper funcional:

```

1  // ANTES      repetido 20+ veces
2  case GO_TO_NEXT_EVIDENCE_CASE -> {
3      try {
4          evidenceCasesNavigationOption("GO_TO_NEXT_EVIDENCE_CASE");
5      } catch (NotEvaluableNetworkException
6              | NonProjectablePotentialException
7              | NotEnoughMemoryException
8              | IncompatibleEvidenceException
9              | CannotNormalizePotentialException
10             | ThereIsNoNextEvidenceCaseException
11             | ThereIsNoPreviousEvidenceCaseException
12             | ConstraintViolatedException ex) {
13          throw new UnrecoverableException(ex);
14      }
15 }
16
17 // DESPU S      un nico wrapper funcional
18 case GO_TO_NEXT_EVIDENCE_CASE ->
19     executeUIAction(() ->
20         evidenceCasesNavigationOption("GO_TO_NEXT_EVIDENCE_CASE"));
21
22 // Implementaci n del wrapper
23 void executeUIAction(UIAction action) {
24     try {
25         action.execute();
26     } catch (OpenMarkovException ex) {
27         throw new UIActionException(ex);
28     }
29 }

```

Listing 7: Simplificación de los bloques `catch` de la GUI con `UIActionException`

4.6 Propuesta F — Corrección de tipos y conflictos de nombres

Impacto: bajo. Riesgo: muy bajo.

Renombrar las ocho clases con errores ortográficos. Al ser en su mayoría clases internas o clases con pocas referencias externas, el impacto es limitado y puede hacerse con herramientas de refactorización automática del IDE (que actualizan todas las referencias en un único paso).

Resolver los conflictos de sombra (`WriterException.NonProjectablePotentialException`, `PGMXParserException.EvidenceIsIncompatibleWithOther`) eliminando las clases internas duplicadas y usando directamente las clases raíz correspondientes.

4.7 Propuesta G — Ocultar excepciones de infraestructura en las APIs de I/O

Impacto: medio. Riesgo: bajo.

Los métodos que hoy declaran `throws SAXException`, `IOException` deben envolver esas excepciones en tipos propios del dominio:

```

1 // ANTES      acopla al llamador con SAX y con java.io
2 public ProbNetReader getProbNetReader(URL url)
3     throws SAXException, IOException,
4           NoReaderForFileException, ParserException;
5
6 // DESPU S    solo tipos del dominio de OpenMarkov
7 public ProbNetReader getProbNetReader(URL url)
8     throws NoReaderForFileException, ParserException;
9 // SAXException e IOException quedan capturadas internamente
10 // y envueltas en ParserException como causa raíz

```

Listing 8: API limpia — sin exposición de detalles de implementación

4.8 Propuesta H — Corrección de la pérdida de traza en `DoEditException`

Impacto: bajo. Riesgo: muy bajo.

En `Node.java` (línea 807), sustituir:

```

1 // ANTES      pierde el stack trace original
2 throw new DoEditException.CannotDoEditException(e.getMessage());
3
4 // DESPU S    preserva la causa completa
5 throw new DoEditException.CannotDoEditException(e);

```

Listing 9: Corrección de la causa raíz perdida en `Node.java`

Esta corrección tiene impacto únicamente en la calidad del diagnóstico: el comportamiento observable para el usuario no cambia, pero los desarrolladores dispondrán de la traza de pila completa cuando investiguen un fallo.

4.9 Propuesta I — Eliminación del re-lanzamiento vacío en `setNetworkType()`

Impacto: bajo. **Riesgo:** muy bajo.

En `ProbNet.java` (líneas 362–368), eliminar el bloque `try/catch` sin tratamiento y dejar que la excepción se propague directamente:

```

1  // ANTES      bloque vac o sin valor a adido
2  public void setNetworkType(NetworkType networkType)
3      throws ConstraintViolatedException {
4      try {
5          // l gica de cambio de tipo
6      } catch (ConstraintViolatedException e) {
7          throw e; // no hace nada
8      }
9  }
10
11 // DESPU S      propagaci n directa, mismo comportamiento
12 public void setNetworkType(NetworkType networkType)
13     throws ConstraintViolatedException {
14     // l gica de cambio de tipo
15     // la excepci n se propaga sola si ocurre
16 }

```

Listing 10: Eliminación del bloque `try/catch` vacío en `ProbNet.java`

4.10 Propuesta J — Revisión de la lógica de navegación de evidencia

Impacto: medio. **Riesgo:** bajo.

Las excepciones `ThereIsNoNextEvidenceCaseException` y `ThereIsNoPreviousEvidenceCaseException` revelan un problema de diseño en la navegación de evidencias. La corrección tiene dos niveles:

1. **Corrección inmediata:** convertir estas excepciones a *unchecked* para eliminar los bloques `catch` redundantes. Ambas se convierten en subclases de `OpenMarkovRuntimeException`, señalando que representan un uso incorrecto del API (intentar navegar más allá del límite sin verificar primero).
2. **Corrección de diseño:** consultar el estado de la lista de evidencias *antes* de habilitar o deshabilitar los botones de navegación. La GUI debe escuchar los cambios en la lista de evidencias y actualizar el estado de los botones en consecuencia, de forma que el usuario nunca pueda pulsar «siguiente» cuando ya está en el último caso.

```

1  // En EvidenceManager o equivalente
2  void onEvidenceCaseListChanged(List<EvidenceCase> cases, int
3      currentIndex) {
4      buttonNext.setEnabled(currentIndex < cases.size() - 1);
5      buttonPrevious.setEnabled(currentIndex > 0);
6      // Las excepciones de "no hay siguiente/anterior" nunca llegar n
7      // a lanzarse
8  }

```

Listing 11: Enfoque preventivo para los botones de navegación de evidencia

5 Plan de migración

La siguiente tabla organiza todas las propuestas en fases de riesgo creciente, de forma que cada fase mejora la situación sin depender de que las fases posteriores estén completas:

#	Cambio	Propuesta	Riesgo
Fase 1 — Inmediata (sin impacto en lógica de negocio)			
1	Corregir 8 typos en nombres de clase	F	Muy bajo
2	Eliminar clases internas con nombre duplicado	F	Muy bajo
3	Corregir <code>catch</code> → <code>null</code> en <code>FormatManager</code>	D	Muy bajo
4	Preservar causa raíz en <code>Node.java</code>	H	Muy bajo
5	Eliminar bloque <code>try/catch</code> vacío en <code>setNetworkType()</code>	I	Muy bajo
Fase 2 — Corto plazo (requiere actualizar los callers)			
6	Convertir <code>a unchecked: InvalidArgumentException</code> → <code>IllegalArgumentExcep-</code>	A	Bajo
7	Convertir <code>a unchecked: NotSupportedOperationException</code> → <code>UnsupportedOperationException</code>	A	Bajo
8	Convertir a <code>unchecked: NoSelectedNodeException</code> , <code>WrongClassException</code> , <code>UnexpectedMenu-</code> <code>ActionException</code>	A	Bajo
9	Convertir a <code>unchecked: ThereIsNoNext/Previous-</code> <code>EvidenceCaseException</code>	J	Bajo
10	Convertir a <code>unchecked: InvalidNetwork-</code> <code>TypeException</code> , <code>CannotNormalize-</code> <code>PotentialException</code> , <code>Unexpected-</code> <code>InferenceException</code>	A	Medio
11	Ocultar <code>SAXException/IOException</code> en APIs de I/O	G	Bajo
Fase 3 — Medio plazo (eliminar throws genéricos)			
12	Especificar <code>throws Exception</code> en <code>LearningAlgorithm</code>	C	Bajo
13	Convertir <code>a unchecked: NonProjectablePotentialException</code> → <code>PotentialComputationException</code>	A+E	Medio
14	Crear <code>UIActionException</code> y consolidar bloques <code>catch</code> de la GUI	J	Medio
15	Revisar lógica de botones de navegación de evidencia (enfoque preventivo)	J	Bajo
Fase 4 — Largo plazo (reestructuración jerárquica completa)			
16	Introducir <code>OpenMarkovException</code> como base <i>checked</i> unificada	E	Alto
17	Introducir <code>UserInputException</code> y <code>NetworkStructureException</code>	E	Alto
18	Introducir <code>OpenMarkovRuntimeException</code> e <code>InternalConsistencyException</code>	E	Alto

19	Migrar	DoEditException:	E	Alto
	ConstraintViolatedException	hereda	de	
	NetworkStructureException	directamente		

Las fases 1 a 3 son completamente independientes entre sí y mejoran la situación de forma incremental. La fase 4 es una refactorización estructural mayor que conviene abordar cuando las capas de aplicación estén estabilizadas y exista cobertura de tests suficiente para verificar que el comportamiento observable no ha cambiado.